

On-Policy Temporal-Difference Control: SARSA(λ)

Reinforcement Learning

Sai Sampath Kedari

sampath@umich.edu

Contents

1	Setup and What Is New	2
2	The λ-Return: Forward View	2
2.1	Definition	2
2.2	The Two Limits	3
2.3	The Forward-View Update	3
3	The Key Identity	4
3.1	Statement	4
3.2	Proof	4
3.3	The Contribution Table	5
3.4	The Forward-View Algorithm	6
4	From Forward View to Backward View: The Column Perspective	6
5	Eligibility Traces	8
5.1	Definition	8
5.2	The Recursive Update	9
5.3	The Backward Update Rule	9
5.4	Verification: Both Views Agree	10

1 Setup and What Is New

We continue with the infinite-horizon discounted MDP of the previous reports: finite state space $\mathcal{S}$, finite action sets $\mathcal{A}(s)$, joint transition kernel $p(s', r \mid s, a)$, discount $\gamma \in [0, 1)$, and stationary policies $\pi(a \mid s)$.

The n -step SARSA report produced, for each depth n , a target

$$q_t^{(n)} = \sum_{i=1}^n \gamma^{i-1} R_{t+i} + \gamma^n Q(S_{t+n}, A_{t+n}),$$

and showed that n is a single knob trading bias against variance: small n bootstraps early (low variance, high bias), large n bootstraps late (high variance, low bias). Choosing a single n forces a single point on that tradeoff.

This report stops choosing. Instead of committing to one depth, it averages all the n -step targets at once, with geometric weights. This introduces *no new target* and *no new sampling*: every $q_t^{(n)}$ was already derived in the n -step report. The only new objects are the weighted average (the λ -return, the forward view of this section) and, in later sections, the eligibility trace that computes that average online. The centerpiece is an identity that rewrites the forward-view correction as a geometrically weighted sum of one-step TD errors; that identity is what makes the online computation possible.

2 The λ -Return: Forward View

2.1 Definition

Rather than committing to a single depth n , SARSA(λ) forms a geometrically weighted average of all n -step targets:

$$q_t^\lambda := (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} q_t^{(n)}, \quad 0 \leq \lambda \leq 1.$$

The weights $(1 - \lambda)\lambda^{n-1}$ are a geometric distribution over the depths $n = 1, 2, 3, \dots$, and they sum to one:

$$(1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} = (1 - \lambda) \cdot \frac{1}{1 - \lambda} = 1.$$

So q_t^λ is a proper convex combination of the n -step targets: a weighted vote in which the 1-step target gets weight $(1 - \lambda)$, the 2-step target $(1 - \lambda)\lambda$, the 3-step target $(1 - \lambda)\lambda^2$, and so on, each successive depth down-weighted by another factor of λ .

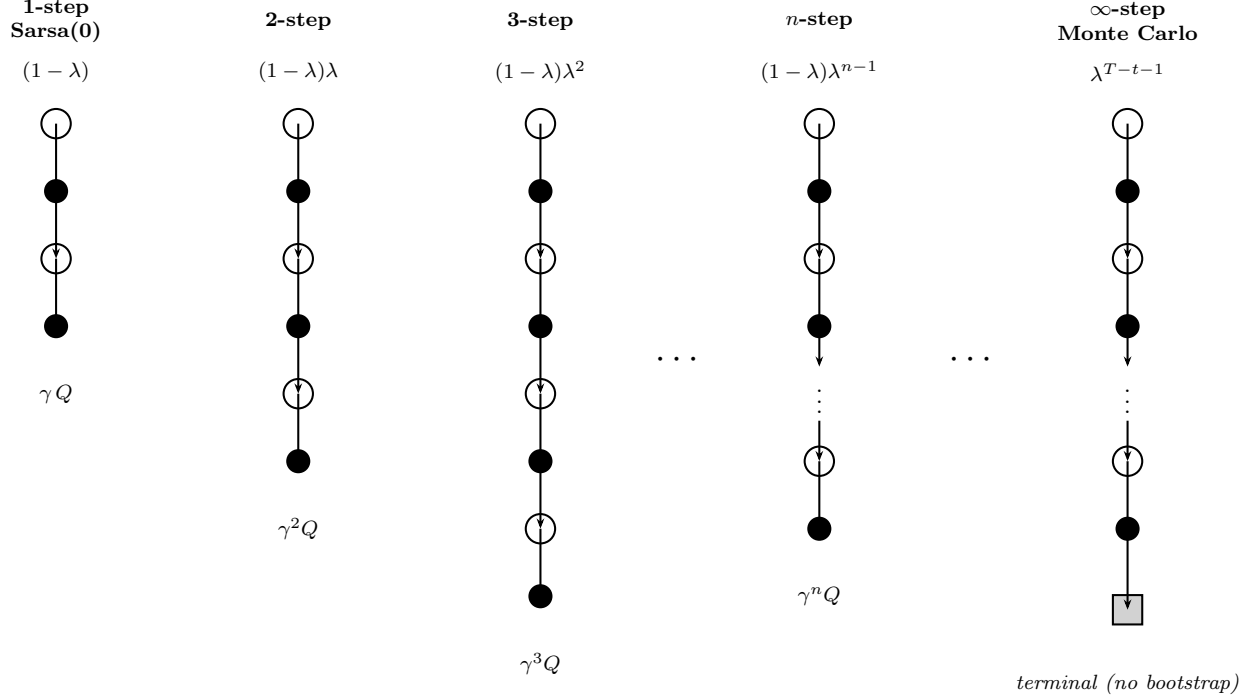


Figure 1: The λ -return as a geometric average of all n -step SARSA backups. Each column is one n -step target from the previous report (empty circles states, filled circles actions, root pair (S_t, A_t) at top). The label above each column is the weight that target receives in the mixture: $(1 - \lambda)\lambda^{n-1}$ for the n -step target, decaying geometrically in n . The final column is the Monte Carlo backup, which runs to the terminal state (gray square) with no bootstrap; in an episode ending at time T it carries the residual weight λ^{T-t-1} , so that all weights sum to one. $\lambda = 0$ places all weight on the 1-step target (SARSA(0)); $\lambda \rightarrow 1$ shifts all weight onto the Monte Carlo target.

2.2 The Two Limits

The single parameter λ slides the mixture between the two extremes of the previous report:

$$\lambda = 0 \implies q_t^\lambda = q_t^{(1)} = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) \quad (\text{SARSA}(0)),$$

since every weight $(1 - \lambda)\lambda^{n-1}$ vanishes for $n \geq 2$ when $\lambda = 0$, leaving only the $n = 1$ term with weight 1. At the other end,

$$\lambda \rightarrow 1 \implies q_t^\lambda \rightarrow G_t \quad (\text{Monte Carlo control}),$$

since the weight slides entirely onto the longest-horizon (full-return) target. Intermediate λ blends all depths at once, rather than picking one.

2.3 The Forward-View Update

The forward-view update steps $Q(S_t, A_t)$ toward the λ -return:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(q_t^\lambda - Q(S_t, A_t)).$$

Everything reduces to the correction $q_t^\lambda - Q(S_t, A_t)$. But this correction depends on all the n -step targets, and the deepest of them require rewards arbitrarily far into the future. So the forward-view

update cannot be applied until the episode ends: it is inherently *offline*. The entire point of the next section is to rewrite this correction in a form that can be computed online, one TD error at a time.

3 The Key Identity

3.1 Statement

Theorem 3.1 (Forward-View Identity for Action Values). *Let $Q : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ be any action-value estimate, and define the one-step TD error*

$$\delta_t := R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t).$$

Then, for any $t \geq 0$,

$$q_t^\lambda - Q(S_t, A_t) = \sum_{k=0}^{\infty} (\gamma\lambda)^k \delta_{t+k}.$$

The forward-view correction at the pair (S_t, A_t) equals a geometrically weighted sum of all future one-step TD errors, with weights $(\gamma\lambda)^k$ decaying at rate $\gamma\lambda$ per step. This is the action-value analogue of the state-value identity proved in the eligibility traces report; the structure of the proof is identical, with the bootstrap $Q(S_{t+k}, A_{t+k})$ replacing $V(S_{t+k})$ throughout.

3.2 Proof

Write the geometric mixture and expand each $q_t^{(n)}$ explicitly:

$$\begin{aligned} q_t^\lambda - Q(S_t, A_t) = (1 - \lambda) \left[\begin{aligned} & R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) \\ & + \lambda(R_{t+1} + \gamma R_{t+2} + \gamma^2 Q(S_{t+2}, A_{t+2})) \\ & + \lambda^2(R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 Q(S_{t+3}, A_{t+3})) \\ & + \lambda^3(R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \gamma^4 Q(S_{t+4}, A_{t+4})) \\ & + \dots \end{aligned} \right] - Q(S_t, A_t). \end{aligned}$$

Collecting the coefficients of each reward term and each bootstrap value:

$$\begin{aligned} q_t^\lambda - Q(S_t, A_t) = (1 - \lambda) \left[\begin{aligned} & R_{t+1}(1 + \lambda + \lambda^2 + \dots) + \gamma R_{t+2}(\lambda + \lambda^2 + \lambda^3 + \dots) + \gamma^2 R_{t+3}(\lambda^2 + \lambda^3 + \dots) + \dots \\ & + \gamma Q(S_{t+1}, A_{t+1}) + \gamma^2 \lambda Q(S_{t+2}, A_{t+2}) + \gamma^3 \lambda^2 Q(S_{t+3}, A_{t+3}) + \dots \end{aligned} \right] - Q(S_t, A_t). \end{aligned}$$

Evaluating each geometric series: $(1 - \lambda)(1 + \lambda + \lambda^2 + \dots) = 1$, $(1 - \lambda)(\lambda + \lambda^2 + \dots) = \lambda$, and in

general $(1 - \lambda)(\lambda^j + \lambda^{j+1} + \dots) = \lambda^j$:

$$\begin{aligned}
q_t^\lambda - Q(S_t, A_t) &= R_{t+1} + (\gamma\lambda)R_{t+2} + (\gamma\lambda)^2R_{t+3} + \dots \\
&\quad + \gamma Q(S_{t+1}, A_{t+1}) + (\gamma\lambda)\gamma Q(S_{t+2}, A_{t+2}) + (\gamma\lambda)^2\gamma Q(S_{t+3}, A_{t+3}) + \dots \\
&\quad - Q(S_t, A_t) - (\gamma\lambda)Q(S_{t+1}, A_{t+1}) - (\gamma\lambda)^2Q(S_{t+2}, A_{t+2}) - \dots \\
&\quad + (\gamma\lambda)Q(S_{t+1}, A_{t+1}) + (\gamma\lambda)^2Q(S_{t+2}, A_{t+2}) + (\gamma\lambda)^3Q(S_{t+3}, A_{t+3}) + \dots \\
&\quad - (\gamma\lambda)Q(S_{t+1}, A_{t+1}) - (\gamma\lambda)^2Q(S_{t+2}, A_{t+2}) - (\gamma\lambda)^3Q(S_{t+3}, A_{t+3}) - \dots
\end{aligned}$$

The last two lines cancel. Grouping the remaining terms into TD errors $\delta_{t+k} = R_{t+k+1} + \gamma Q(S_{t+k+1}, A_{t+k+1}) - Q(S_{t+k}, A_{t+k})$:

$$\begin{aligned}
q_t^\lambda - Q(S_t, A_t) &= \underbrace{(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t))}_{\delta_t} \\
&\quad + (\gamma\lambda) \underbrace{(R_{t+2} + \gamma Q(S_{t+2}, A_{t+2}) - Q(S_{t+1}, A_{t+1}))}_{\delta_{t+1}} \\
&\quad + (\gamma\lambda)^2 \underbrace{(R_{t+3} + \gamma Q(S_{t+3}, A_{t+3}) - Q(S_{t+2}, A_{t+2}))}_{\delta_{t+2}} \\
&\quad + \dots
\end{aligned}$$

Key Result

$$q_t^\lambda - Q(S_t, A_t) = \sum_{k=0}^{\infty} (\gamma\lambda)^k \delta_{t+k}.$$

The forward-view correction equals a geometrically weighted sum of all future one-step TD errors, with weights $(\gamma\lambda)^k$ decaying at rate $\gamma\lambda$ per step.

This completes the proof. $\square$

3.3 The Contribution Table

It is useful to lay out all these coefficients explicitly. Each row corresponds to a visited state-action pair, each column to a TD error, and entry (τ, t) is the coefficient with which δ_t contributes to the update of $Q(S_\tau, A_\tau)$, as given directly by the key identity.

	δ_1	δ_2	δ_3	δ_4	δ_5	$\delta_6 \dots$
$Q(S_1, A_1)$	1	$\gamma\lambda$	$(\gamma\lambda)^2$	$(\gamma\lambda)^3$	$(\gamma\lambda)^4$	$(\gamma\lambda)^5 \dots$
$Q(S_2, A_2)$	0	1	$\gamma\lambda$	$(\gamma\lambda)^2$	$(\gamma\lambda)^3$	$(\gamma\lambda)^4 \dots$
$Q(S_3, A_3)$	0	0	1	$\gamma\lambda$	$(\gamma\lambda)^2$	$(\gamma\lambda)^3 \dots$
$Q(S_4, A_4)$	0	0	0	1	$\gamma\lambda$	$(\gamma\lambda)^2 \dots$
$Q(S_5, A_5)$	0	0	0	0	1	$\gamma\lambda \dots$
$Q(S_6, A_6)$	0	0	0	0	0	1 $\dots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

Table 1: Coefficient with which δ_t contributes to the update of $Q(S_\tau, A_\tau)$. Entry (τ, t) equals $(\gamma\lambda)^{t-\tau}$ when $t \geq \tau$, and 0 when $t < \tau$. The diagonal is all 1: a pair always receives its own TD error in full.

Each row is the forward-view identity written out for that pair. Row $Q(S_3, A_3)$, for instance, reads: update $Q(S_3, A_3)$ using δ_3 with weight 1, then δ_4 with weight $\gamma\lambda$, then δ_5 with weight $(\gamma\lambda)^2$, and so on. The zeros to the left of the diagonal reflect the fact that $Q(S_3, A_3)$ has no dependence on δ_1 or δ_2 , which were observed before the pair (S_3, A_3) was visited.

3.4 The Forward-View Algorithm

The forward view at each time t proceeds as follows:

1. Run the episode to completion and observe the full trajectory from (S_t, A_t) onward.
2. Compute all future TD errors $\delta_t, \delta_{t+1}, \delta_{t+2}, \dots$
3. Form the weighted sum $\sum_{k=0}^{\infty} (\gamma\lambda)^k \delta_{t+k}$.
4. Apply a single update to $Q(S_t, A_t)$.
5. Repeat for every visited pair $t = 1, 2, \dots, T$.

This is the row-wise computation in Table 1: for each row, scan all columns to the right of the diagonal and accumulate. Because every update requires future information, the forward view is inherently offline. No update to any $Q(S_t, A_t)$ can be made until the episode ends.

4 From Forward View to Backward View: The Column Perspective

Return to Table 1 and focus on a single entry: the coefficient $(\gamma\lambda)^2$ sitting at row $Q(S_1, A_1)$, column δ_3 .

Ask yourself: at time $t = 3$, the moment δ_3 is computed, do we already know this coefficient? Yes, completely. It is $(\gamma\lambda)^{3-1} = (\gamma\lambda)^2$, determined entirely by the fact that (S_1, A_1) was visited at time 1 and we are currently at time 3. It does not depend on δ_4 , δ_5 , or anything that happens in the future.

Now look at the entire column δ_3 . Every entry in that column is known at time $t = 3$:

- $Q(S_1, A_1)$ should receive δ_3 with weight $(\gamma\lambda)^2$, because (S_1, A_1) was visited 2 steps ago.
- $Q(S_2, A_2)$ should receive δ_3 with weight $(\gamma\lambda)^1 = \gamma\lambda$, because (S_2, A_2) was visited 1 step ago.
- $Q(S_3, A_3)$ should receive δ_3 with weight $(\gamma\lambda)^0 = 1$, because (S_3, A_3) is the current pair.
- $Q(S_4, A_4), Q(S_5, A_5), \dots$ receive weight 0, because they have not been visited yet.

Every number in column δ_3 is determined at time $t = 3$.

This is the central observation. The forward view processes the table row by row: for each pair (S_τ, A_τ) , scan rightward and wait for all future TD errors. But the entries in the table are not determined by the future. The coefficient $(\gamma\lambda)^{t-\tau}$ depends only on how long ago (S_τ, A_τ) was visited. No future information is involved at all.

The forward view is waiting for something it does not actually need to wait for.

The alternative strategy. Instead of processing the table row by row, process it column by column. At time t , the moment δ_t is observed, apply its contribution to every previously visited pair immediately:

$$Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha \cdot \delta_t \cdot (\gamma\lambda)^{t-\tau}, \quad \tau = 1, 2, \dots, t.$$

Then at time $t + 1$, when δ_{t+1} arrives, apply column δ_{t+1} immediately. And so on, step by step throughout the episode.

By the end of the episode, what has each $Q(S_\tau, A_\tau)$ received in total? It has received, one at a time as each TD error arrived:

$$\delta_\tau \cdot 1, \quad \delta_{\tau+1} \cdot \gamma\lambda, \quad \delta_{\tau+2} \cdot (\gamma\lambda)^2, \quad \dots$$

The total accumulated update is

$$\sum_{k=0}^{\infty} (\gamma\lambda)^k \delta_{\tau+k},$$

which is exactly the forward-view correction $q_\tau^\lambda - Q(S_\tau, A_\tau)$.

The table is the same. The numbers are the same. The total updates are identical. The only thing that changed is the order of computation: rows versus columns.

Key Result

Forward view (row-wise, offline): fix a pair (S_τ, A_τ) , wait for all future TD errors, then apply a single update:

$$Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha \sum_{k=0}^{\infty} (\gamma\lambda)^k \delta_{\tau+k}.$$

One pair is updated. The entire future trajectory is required.

Backward view (column-wise, online): at each step t , the moment δ_t arrives, update every previously visited pair immediately:

$$Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha \cdot \delta_t \cdot (\gamma\lambda)^{t-\tau}, \quad \forall \tau \leq t.$$

All past pairs are updated. No future information is required.

The backward view is not an approximation of the forward view. It is the forward view, computed in a different order. The coefficients $(\gamma\lambda)^{t-\tau}$ are determined entirely by the past, by when each pair was last visited, and are therefore available online the moment δ_t is computed. Computing them directly from the definition would cost $O(t)$ work per step, since it iterates over the entire visited history. We do not need to store that history, however: for each pair (s, a) , it suffices to maintain a single running weight accumulating the coefficient $(\gamma\lambda)^{t-\tau}$ from all past visits. That is what eligibility traces do.

5 Eligibility Traces

5.1 Definition

At time t , the *eligibility trace* of the state-action pair (s, a) is defined as

$$E_t(s, a) := \sum_{\tau=1}^t (\gamma\lambda)^{t-\tau} \mathbf{1}\{S_\tau = s, A_\tau = a\}, \quad E_0(s, a) = 0 \quad \forall (s, a).$$

This is exactly the entry in column δ_t of Table 1 corresponding to the pair (s, a) : for every past time τ at which the trajectory visited (s, a) , we include the contribution $(\gamma\lambda)^{t-\tau}$, decaying geometrically with the number of steps elapsed since that visit.

To make this concrete, consider the trajectory at time $t = 5$. The eligibility traces are:

$$\begin{aligned} E_5(S_1, A_1) &= (\gamma\lambda)^4, \\ E_5(S_2, A_2) &= (\gamma\lambda)^3, \\ E_5(S_3, A_3) &= (\gamma\lambda)^2, \\ E_5(S_4, A_4) &= (\gamma\lambda)^1 = \gamma\lambda, \\ E_5(S_5, A_5) &= (\gamma\lambda)^0 = 1. \end{aligned}$$

These are exactly the entries in column δ_5 of Table 1, read top to bottom. The eligibility trace table $\{E_t(s, a)\}_{(s, a)}$ stores the entire current column of the table in a single array indexed by state-action pair.

5.2 The Recursive Update

Computing $E_t(s, a)$ from scratch at each step via the definition above requires iterating over the full history at cost $O(t)$. We now derive a one-step recursion that reduces this to $O(|\mathcal{S}||\mathcal{A}|)$.

Peel off the $\tau = t$ term from the definition:

$$\begin{aligned}
E_t(s, a) &= \sum_{\tau=1}^t (\gamma\lambda)^{t-\tau} \mathbf{1}\{S_\tau = s, A_\tau = a\} \\
&= (\gamma\lambda)^0 \mathbf{1}\{S_t = s, A_t = a\} + \sum_{\tau=1}^{t-1} (\gamma\lambda)^{t-\tau} \mathbf{1}\{S_\tau = s, A_\tau = a\} \\
&= \mathbf{1}\{S_t = s, A_t = a\} + (\gamma\lambda) \sum_{\tau=1}^{t-1} (\gamma\lambda)^{(t-1)-\tau} \mathbf{1}\{S_\tau = s, A_\tau = a\} \\
&= \mathbf{1}\{S_t = s, A_t = a\} + (\gamma\lambda) E_{t-1}(s, a).
\end{aligned}$$

Therefore the eligibility trace satisfies the recursion:

Key Result

$$E_t(s, a) = \gamma\lambda E_{t-1}(s, a) + \mathbf{1}\{S_t = s, A_t = a\}, \quad E_0(s, a) = 0.$$

The interpretation in terms of Table 1 is immediate. Moving from column $t-1$ to column t : every existing entry is multiplied by $\gamma\lambda$, because every past visit is now one step older and therefore one factor of $\gamma\lambda$ more decayed; and a fresh 1 is added at the current pair (S_t, A_t) , because it has just been visited. The recursion builds column t from column $t-1$ in a single pass over all pairs, at cost $O(|\mathcal{S}||\mathcal{A}|)$ per step.

The two limits of λ read off the recursion directly. When $\lambda = 0$ the trace collapses to $E_t(s, a) = \mathbf{1}\{S_t = s, A_t = a\}$: only the current pair is eligible, δ_t updates only $Q(S_t, A_t)$, and we recover SARSA(0). When $\lambda = 1$ the trace becomes $E_t(s, a) = \sum_{\tau=1}^t \gamma^{t-\tau} \mathbf{1}\{S_\tau = s, A_\tau = a\}$: every past visit stays eligible, weighted only by the MDP discount γ , approaching the full Monte Carlo update.

5.3 The Backward Update Rule

With eligibility traces in hand, the backward-view update at each time step t is:

1. Observe R_{t+1} , S_{t+1} , draw $A_{t+1} \sim \pi(\cdot | S_{t+1})$, and compute $\delta_t = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)$.
2. Update every trace: $E_t(s, a) \leftarrow \gamma\lambda E_{t-1}(s, a) + \mathbf{1}\{S_t = s, A_t = a\}$ for all (s, a) .
3. Update every estimate: $Q(s, a) \leftarrow Q(s, a) + \alpha \delta_t E_t(s, a)$ for all (s, a) .

Steps 2 and 3 together cost $O(|\mathcal{S}||\mathcal{A}|)$ per time step, independent of the episode length. In practice, pairs with $E_t(s, a) \approx 0$ receive negligible updates and can be skipped, making the algorithm efficient even for large state-action spaces.

5.4 Verification: Both Views Agree

We verify algebraically that the total update accumulated by the backward view equals the forward-view target. The backward view applies $\alpha \delta_t E_t(S_\tau, A_\tau)$ to $Q(S_\tau, A_\tau)$ at each time t . The total update across the episode is proportional to

$$\sum_{t=\tau}^{\infty} \delta_t \cdot E_t(S_\tau, A_\tau).$$

Assuming (S_τ, A_τ) is visited once at time τ , we have $E_t(S_\tau, A_\tau) = (\gamma\lambda)^{t-\tau}$ for all $t \geq \tau$. If the pair is visited at multiple times, the trace accumulates a contribution from each visit, and by linearity of the update the same argument applies term by term. Therefore:

$$\sum_{t=\tau}^{\infty} \delta_t \cdot E_t(S_\tau, A_\tau) = \sum_{t=\tau}^{\infty} (\gamma\lambda)^{t-\tau} \delta_t = \sum_{k=0}^{\infty} (\gamma\lambda)^k \delta_{\tau+k},$$

which is exactly the forward-view correction $q_\tau^\lambda - Q(S_\tau, A_\tau)$ from Theorem 3.1. The two views are algebraically identical.

Key Result

Eligibility traces $E_t(s, a) = \gamma\lambda E_{t-1}(s, a) + \mathbf{1}\{S_t = s, A_t = a\}$ store the column weights of Table 1 online and recursively, at cost $O(|\mathcal{S}||\mathcal{A}|)$ per step. The update $Q(s, a) \leftarrow Q(s, a) + \alpha \delta_t E_t(s, a)$ applies each column of the table immediately when δ_t is observed. Summing these online updates across the episode recovers exactly the forward-view target $q_\tau^\lambda - Q(S_\tau, A_\tau)$ for every pair. Backward SARSA(λ) is the online, efficient implementation of forward SARSA(λ).